

Вопросы

1. Какие проблемы могут возникнуть при отсутствии синхронизации между взаимосвязанными потоками?
2. Почему операция Count++ не является атомарной?
3. Что происходит при перемешивании (interleaving) потоков в мультипрограммной системе?
4. Что такое гонки (условия состязания) и как они связаны с проблемой синхронизации?
5. Как можно обеспечить взаимное исключение (mutual exclusion) в критических секциях потоков?
6. Что такое спин-блокировка и в каких случаях она может быть полезна?
7. Какие функции API ОС Windows используются для организации входа и выхода из критической секции потоков одного процесса в режиме пользователя?
8. Чем отличаются критические секции от объектов ядра при синхронизации потоков?
9. Каким образом потоки могут ожидать освобождения объекта ядра и продолжить выполнение с использованием функции WaitForSingleObject()?
10. Чем отличаются мьютексы от семафоров в контексте синхронизации потоков разных процессов?
11. Какие функции используются для инициализации мьютекса, входа в критическую секцию и выхода из нее?
12. Какие задачи решает объект "событие" в многопоточных программных системах?
13. Как создать событие и какими функциями управлять его состоянием?
14. Какие другие объекты ядра могут быть использованы для синхронизации потоков, помимо мьютексов и событий?
15. Какие особенности синхронизации возникают при работе с ядром операционной системы Windows и почему требуется использование спин-блокировок?

Словарь

1. В качестве TSL инструкции, необходимой для решения проблемы входа в критическую секцию, можно использовать функцию InterlockedCompareExchange.
2. Для синхронизации потоков содержатся сведения о различных объектах ядра во многих руководствах по программированию и в MSDN. Например, процесс и поток находятся в состоянии активности или завершения, а файл может быть в занятом или свободном состоянии в зависимости от операции ввода-вывода. Также

- уведомление об изменении файла может быть в занятом или свободном состоянии в зависимости от наличия изменений в файловой системе.
3. Для организации входа и выхода из критической секции потоков одного процесса в режиме пользователя используются специальные и эффективные функции `EnterCriticalSection` и `LeaveCriticalSection`, принимающие предварительно проинициализированную структуру `CRITICAL_SECTION` в качестве параметра.
 4. Многочисленные попытки входа в критическую секцию могут быть интересными при использовании внешнего цикла. Синхронизацию единичных попаданий можно обеспечить и другими средствами.
 5. Для ожидания освобождения объекта и приостановки потока используется функция `WaitForSingleObject`, принимающая описатель ожидаемого объекта ядра и максимальное время ожидания в миллисекундах.
 6. С помощью функций `Open` поток с нужными правами доступа получает описатель объекта, зная его имя.
 7. Семафоры, предложенные Дейкстрой, представляют собой целочисленную переменную в пространстве ядра, доступ к которой осуществляется через атомарные операции `wait` и `signal`.
 8. Для корректного приращения переменной `Count` можно использовать операцию `InterlockedExchangeAdd(&Count, 1)`, как описано в начале лекции.
 9. Пролог (`entry section`) является наиболее важным с точки зрения синхронизации, так как здесь принимается решение о допуске потока в критическую секцию.
 10. Объекты ядра могут иметь имя и позволяют задавать тайм-аут для времени ожидания, а также обладают другими возможностями для гибкой реализации синхронизации.
 11. Наиболее примитивными объектами ядра являются "события". Они используются для информирования одного потока другим о завершении операции.
 12. Перед входом в критическую секцию поток проверяет возможность входа и, если это невозможно, продолжает опрашивать значение переменной-замка.
 13. Поток создает объект ядра при помощи функций семейства `Create`, например, `CreateSemaphore` или `CreateThread`. Затем объект становится доступным для всех потоков данного процесса с помощью его описателя.
 14. Для синхронизации в программе `async` можно использовать перечисленные примитивы.
 15. Гонки (`race conditions`) возникают, когда два или более потоков обрабатывают разделяемые данные, и конечный результат зависит от относительной скорости выполнения этих процессов.
 16. Временная спин-блокировка может быть полезна на многопроцессорных машинах, где один поток находится в цикле, а

другой работает на другом процессоре и может изменить значение переменной-замка.

17. Interlocked-функции выполняются в пользовательском режиме процессора и очень быстро, примерно за 50 тактов.
18. Функции EnterCriticalSection и LeaveCriticalSection реализованы на основе Interlocked-функций, выполняются атомарно и работают очень быстро.
19. Функция InterlockedExchangeAdd позволяет атомарно увеличить значение переменной. Она принимает указатель на переменную и значение приращения.
20. Открытие существующего объекта по имени является одним из способов работы с объектами ядра, поскольку многие из них имеют имена в пространстве имен объектов. Имя объекта является одним из параметров функций Create.